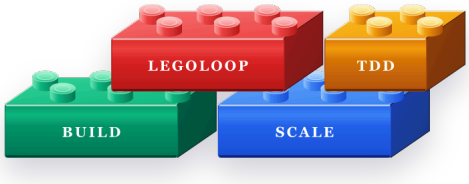
Manual Didático Corporativo LegoLoop



SOBRE ESTA OBRA

Documento gerado deterministicamente a partir das fontes citadas no próprio texto, via pandoc + typst. Toda afirmação técnica desta obra é rastreável a um arquivo nomeado no capítulo correspondente.

Data da compilação: · LegoLoop Foundation

AI Motion Graphics — Livro do Engenheiro

> ***Público:*** *Desenvolvedores, arquitetos de software, operadores LegoLoop*
> ***Nível:*** *Avançado* · ***Data:*** *2026-11-09* · ***Versão:*** *1.0.0*

Sumário

1. Fundamentos — O Problema e a Solução
2. Os 10 Repos: Análise Técnica
3. Arquitetura do Pipeline DAG
4. Motor de Seleção Híbrida
5. Renderização Playwright + FFmpeg
6. Sync de Áudio com Librosa
7. Operação e Solução de Problemas

Capítulo 1 — Fundamentos: O Problema e a Solução

1.1 O Custo do "AI Slop"

Motion graphics gerados por IA sem ferramentas especializados produzem visual genérico — o *AI slop*. A evidência empírica demonstra que repos especializados elevam a qualidade para profissional.

A tese: **qualidade vem de ativos curados por humanos combinados com seleção e execução por IA**.

1.2 Os 3 Cenários

Cenário	Prompt	Repos Primários	Output
Explainer	"Explique painel solar"	Three.js + Postprocessing	3D infográfico
Lançamento	"Lançamento do app X"	Video ShotCraft + Librosa	Promo sincronizada
Pixel Art	"Tempestade pixel art"	PixelFX + Lemo Opuscar	Animação retrô

1.3 Soberania

Todos os 10 repos são open-source (MIT/ISC). Custo mensal: **R$ 0,00**.

Capítulo 2 — Os 10 Repos: Análise Técnica

Tier 1 — Essenciais

Three.js (MIT, 100k+ stars)
Renderização 3D via WebGL. Bar charts, modelos procedurais, partículas, fly-throughs.

PixelFX (Open-source)
Pixel art com biblioteca de efeitos (explosões, clima, relâmpagos).

Video ShotCraft (Open-source)
124 templates de promo/lançamento. Seleção automática.

Librosa (ISC, v0.11.0 instalado)
Beat detection ±50ms. Sincronização visual-musical.

React Postprocessing (MIT)
Glow, DoF, film grain, lens coloring, glitch.

Tier 2 — Expansão

Repo	Gatilho	Capacidade
Lemo Opuscar	Estilo cinematográfico	66 estilos
Blender MCP	CGI profissional	Cenas 3D via MCP
Prompt Packs	Efeitos prontos	Prompts testados
Claude Animation	Mascote explicativo	Clawd + emotions
React Rapier	Simulação física	Jenga, dominós

Capítulo 3 — Arquitetura do Pipeline DAG

3.1 Topologia

```
TICK-01.1 → TICK-01.2 → TICK-01.3 → SG-E01 →
TICK-02.1 → TICK-02.2 → SG-E02 →
TICK-03.1 → TICK-03.2 → TICK-03.3 → SG-E03 →
TICK-04.1 → TICK-04.2 → SG-E04 →
TICK-05.1 → TICK-05.2 → SG-FINAL
```

3.2 Contratos

```go
var ErrInvalidRequest = errors.New("motion: requisição inválida")
var ErrNoRepoSelected = errors.New("motion: nenhum repo selecionado")

type Request struct {
    Prompt     string   `json:"prompt"`
    Scenario   Scenario `json:"scenario"`
    Resolution string   `json:"resolution"`
    Duration   float64  `json:"duration"`
}
```

3.3 Costuras de Teste

Costura	Tipo	Justificativa
`motion.Run(req)`	Primária	Contrato externo
`frameCapturer`	Interface	Separa Playwright
`frameEncoder`	Interface	Separa FFmpeg
`beatDetector`	Interface	Separa Librosa

Capítulo 4 — Motor de Seleção Híbrida

4.1 Algoritmo

1. Validar scenario (enum check)
2. Mapear scenario → lista ordenada de repos
3. Atribuir confidence do catálogo
4. Gerar Reason contextual
5. Retornar lista rankeada

4.2 Mapeamento

Cenário	Repos (prioridade)
explainer	threejs(0.90) → react-postprocessing(0.80) → librosa(0.85)
pixel_art	pixelfx(0.95) → lemo-opuscar(0.75)
product_launch	video-shotcraft(0.90) → librosa(0.85) → threejs(0.90)

4.3 Fluxo HITL

1. Pipeline sugere repos
2. Operador confirma ou ajusta
3. Repos confirmados → BuildHTML()
4. Pular confirmação = violação de guardrail

Capítulo 5 — Renderização Playwright + FFmpeg

5.1 Playwright

```
captureFramesWith(capturer, html, fps, duration, resolution, outDir)
→ N = fps × duration frames PNG
→ 90 frames para 3s @ 30fps
```

5.2 FFmpeg

```
ffmpeg -y -framerate 30 -i frame_%04d.png \
  -c:v libx264 -pix_fmt yuv420p -s 1920x1080 -r 30 output.mp4
```

5.3 Verificação

```
ffprobe -show_entries stream=codec_name,width,height output.mp4
# Esperado: h264, 1920x1080
```

5.4 Limites

Métrica	Limite
Renderização 60s	≤ 300s
HTML máximo	50KB
FPS	30

Capítulo 6 — Sync de Áudio com Librosa

6.1 Pipeline

```
detectBeatsWith(detector, audioPath) → []float64

1. Validar audioPath
2. Verificar existência
3. Invocar Librosa
4. Retornar timestamps
5. Fallback gracioso → nil
```

6.2 Script Python

```python
import librosa
y, sr = librosa.load(audio_path, sr=22050)
tempo, beats = librosa.beat.beat_track(y=y, sr=sr)
return [float(b * 512 / sr) for b in beats]
```

6.3 Precisão

- 120 BPM: beats a cada 0.5s, tolerância ±50ms
- Mínimo: ≥ 4 beats para 3s
- Fallback: áudio ausente → sem sync

Capítulo 7 — Operação e Solução de Problemas

Comandos

```
go test -count=1 ./internal/motion/...
go vet ./internal/motion/...
gofmt -l internal/motion/
go build -o bin/legoloop.exe ./cmd/legoloop
legoloop validate loops/ai-motion-graphics/pipeline.dag.json
```

Problemas Comuns

Erro	Solução
ErrInvalidRequest	Validar entrada
ErrNoRepoSelected	Confirmar seleção
Playwright não inicia	npx playwright install chromium
FFmpeg não encontrado	winget install ffmpeg
MP4 sem áudio	Incluir AudioPath

Portões de Qualidade

Portão	Critério
Testes	32/32, exit 0
Estática	0 warnings
Formatação	saída vazia
Build	exit 0
DAG	exit 0